

ADS Practical No. 1

a)1-d arrays like - sum of elements of array, searching an element in array, finding minimum and maximum element in array

Using normal list as array

```
def array_creation():  
    no_of_array_elements = int(input("Enter the number of array elements you  
want: "))  
    array = []  
    for i in range(no_of_array_elements):  
        elements = int(input( f"Enter the {i} element: "))  
        array.append(elements)  
    return array
```

```
array = array_creation()
```

```
while True:
```

```
    print("\n1-D array functions")  
    print("1. Sum of elements of 1-d array")  
    print("2. Searching an element in a 1-d array")  
    print("3. Finding minimum and maximum element in a 1-d array")  
    print("4. Exit")
```

```
choice = int(input("Choose what option you would like to perform: "))
```

```
if choice == 1:
```

```
    print('The sum of the array is:', sum(array))
```

```
elif choice == 2:
```

```
search_element = int(input('Enter the element you want to search: '))
for i in array:
    if i == search_element:
        print(f"Element exists in the array in the index
{array.index(search_element)}")
        break
    else:
        print("Element doesn't exist in the array")

elif choice == 3:
    print("The maximum element in the array is: ", max(array))
    print("The minimum element in the array is: ", min(array))

elif choice == 4:
    print("Goodbye")
    break

else:
    print("Please enter a valid input")

continue_choice = input("\nDo you want to continue (y/n): ")
if continue_choice != 'y':
    print("Goodbye")
    break
```

OUTPUT-

```
=== RESTART: C:\SYDSDA\ADS(Algorithm and data structure)\Practi
Enter the number of array elements you want: 5
Enter the 0 element: 1
Enter the 1 element: 2
Enter the 2 element: 3
Enter the 3 element: 4
Enter the 4 element: 5
```

1-D array functions

1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit

Choose what option you would like to perform: 1

The sum of the array is: 15

Do you want to continue (y/n): y

1-D array functions

1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit

Choose what option you would like to perform: 2

Enter the element you want to search: 4

Element exists in the array in the index 3

Do you want to continue (y/n): y

1-D array functions

1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit

Choose what option you would like to perform: 3

The maximum element in the array is: 5

The minimum element in the array is: 1

Do you want to continue (y/n): y

1-D array functions

1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit

Choose what option you would like to perform: 4

Goodbye

Using numpy to create array

```
import numpy as np

def array_creation():
    no_of_array_elements = int(input("Enter the number of array elements you want: "))
    array = []
    for i in range(no_of_array_elements):
        elements = int(input(f"Enter the {i} element: "))
        array.append(elements)
    return np.array(array)

array = array_creation()
```

```
while True:
    print("\n1-D array functions")
    print("1. Sum of elements of 1-d array")
    print("2. Searching an element in a 1-d array")
    print("3. Finding minimum and maximum element in a 1-d array")
    print("4. Exit")
```

```
choice = int(input("Choose what option you would like to perform: "))
```

```
if choice == 1:
    print('The sum of the array is:', np.sum(array))
```

```
elif choice == 2:
```

```
search_element = int(input('Enter the element you want to search: '))
if search_element in array:
    index = np.argmax(array == search_element)
    print(f"Element exists in the array at index {index}")
else:
    print("Element doesn't exist in the array")

elif choice == 3:
    print("The maximum element in the array is:", np.max(array))
    print("The minimum element in the array is:", np.min(array))

elif choice == 4:
    print("Goodbye")
    break

else:
    print("Please enter a valid input")

continue_choice = input("Do you want to continue (y/n): ")
if continue_choice != 'y':
    print("Goodbye")
    break
```

OUTPUT-

```
=== RESTART: C:\SYDSDA\ADS(Algorithm and data structure)\Practic
Enter the number of array elements you want: 4
Enter the 0 element: 9
Enter the 1 element: 7
Enter the 2 element: 5
Enter the 3 element: 3
```

```
1-D array functions
1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit
Choose what option you would like to perform: 1
The sum of the array is: 24
Do you want to continue (y/n): y
```

```
1-D array functions
1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit
Choose what option you would like to perform: 2
Enter the element you want to search: 5
Element exists in the array at index 2
Do you want to continue (y/n): y
```

```
1-D array functions
1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit
Choose what option you would like to perform: 3
The maximum element in the array is: 9
The minimum element in the array is: 3
Do you want to continue (y/n): y
```

```
1-D array functions
1. Sum of elements of 1-d array
2. Searching an element in a 1-d array
3. Finding minimum and maximum element in a 1-d array
4. Exit
Choose what option you would like to perform: 4
Goodbye
|
```

b)Programs on 2-d arrays like row-sum, column-sum, addition of two matrices ,multiplication of two matrices

#using list to create 2d_array

```
def create_2d_list_array():
    rows = int(input("Enter the number of rows: "))
    cols = int(input("Enter the number of columns: "))
    print("Enter the elements row-wise:")
    list_array = []
    for i in range(rows):
        row = []
        for j in range(cols):
            element = int(input(f"Enter element at position [{i}][{j}]: "))
            row.append(element)
        list_array.append(row)
    return list_array

def row_sum(list_array):
    result = []
    for row in list_array:
        result.append(sum(row))
    return result

def column_sum(list_array):
    if not list_array:
        return []
    cols = len(list_array[0])
```

```
result = []
for j in range(cols):
    col_total = 0
    for i in range(len(list_array)):
        col_total += list_array[i][j]
    result.append(col_total)
return result
```

```
def matrix_addition(A, B):
    result = []
    for i in range(len(A)):
        row = []
        for j in range(len(A[0])):
            row.append(A[i][j] + B[i][j])
        result.append(row)
    return result
```

```
def matrix_multiplication(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    cols_B = len(B[0])
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
```



```
return result
```

```
def get_matrix_shape(list_array):  
    return len(list_array), len(list_array[0]) if list_array else 0
```

```
print("Enter the main matrix:")  
main_matrix = create_2d_list_array()
```

```
while True:  
    print("\n2-D List Operations")  
    print("1. Row-wise Sum")  
    print("2. Column-wise Sum")  
    print("3. Addition with Another Matrix")  
    print("4. Multiplication with Another Matrix")  
    print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:  
    print("Row-wise sum:", row_sum(main_matrix))
```

```
elif choice == 2:  
    print("Column-wise sum:", column_sum(main_matrix))
```

```
elif choice == 3:  
    print("Enter another matrix for addition:")
```

```
matrix2 = create_2d_list_array()
if get_matrix_shape(main_matrix) == get_matrix_shape(matrix2):
    result = matrix_addition(main_matrix, matrix2)
    print("Result after addition:")
    for row in result:
        print(row)
else:
    print("Error: Matrices must be the same size.")

elif choice == 4:
    print("Enter another matrix for multiplication:")
    matrix2 = create_2d_list_array()
    if get_matrix_shape(main_matrix)[1] == get_matrix_shape(matrix2)[0]:
        result = matrix_multiplication(main_matrix, matrix2)
        print("Result after multiplication:")
        for row in result:
            print(row)
    else:
        print("Error: Columns of first matrix must equal rows of second.")

elif choice == 5:
    print("Goodbye!")
    break

else:
    print("Invalid choice. Try again.")
```

```
cont = input("Do you want to continue? (y/n): ").lower()
```

```
if cont != 'y':
```

```
    print("Goodbye!")
```

```
    break
```

#OUTPUT

```
=== RESTART: C:\SYDSDA\ADS (Algorithm and (
Enter the main matrix:
Enter the number of rows: 2
Enter the number of columns: 3
Enter the elements row-wise:
Enter element at position [0][0]: 1
Enter element at position [0][1]: 2
Enter element at position [0][2]: 4
Enter element at position [1][0]: 6
Enter element at position [1][1]: 8
Enter element at position [1][2]: 4

2-D List Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
Enter your choice: 1
Row-wise sum: [7, 18]
Do you want to continue? (y/n): y

2-D List Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
Enter your choice: 2
Column-wise sum: [7, 10, 8]
Do you want to continue? (y/n): y
```

```
2-D List Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
```

```
Enter your choice: 3
Enter another matrix for addition:
Enter the number of rows: 2
Enter the number of columns: 3
Enter the elements row-wise:
Enter element at position [0][0]: 7
Enter element at position [0][1]: 4
Enter element at position [0][2]: 9
Enter element at position [1][0]: 3
Enter element at position [1][1]: 1
Enter element at position [1][2]: 6
Result after addition:
[8, 6, 13]
[9, 9, 10]
Do you want to continue? (y/n): y
```

```
2-D List Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
```

```
Enter your choice: 4
Enter another matrix for multiplication:
Enter the number of rows: 3
Enter the number of columns: 2
Enter the elements row-wise:
Enter element at position [0][0]: 8
Enter element at position [0][1]: 4
Enter element at position [1][0]: 6
Enter element at position [1][1]: 2
Enter element at position [2][0]: 6
Enter element at position [2][1]: 4
Result after multiplication:
[44, 24]
[120, 56]
Do you want to continue? (y/n): y
```

```
2-D List Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
Enter your choice: 5
```

Using numpy to create 2D_array

```
import numpy as np
```

```
def create_2d_array():
```

```
    rows = int(input("Enter the number of rows: "))
```

```
    cols = int(input("Enter the number of columns: "))
```

```
    print("Enter the elements row-wise:")
```

```
    array = []
```

```
    for i in range(rows):
```

```
        row = []
```

```
        for j in range(cols):
```

```
            element = int(input(f"Enter element at position [{i}][{j}]: "))
```

```
            row.append(element)
```

```
        array.append(row)
```

```
    return np.array(array)
```

```
def row_sum(matrix):
```

```
    result = []
```

```
    m, n = matrix.shape
```

```
    for i in range(m):
```

```
        total = 0
```

```
        for j in range(n):
```

```
            total += matrix[i][j]
```

```
        result.append(total)
```

```
    return result
```

```
def column_sum(matrix):
```

```
    result = []
```

```
    m, n = matrix.shape
```

```
    for j in range(n):
```

```
        total = 0
```

```
        for i in range(m):
```

```
            total += matrix[i][j]
```

```
        result.append(total)
```

```
    return result
```

```
def matrix_addition(A, B):
```

```
    result = []
```

```
    for i in range(len(A)):
```

```
        row = []
```

```
        for j in range(len(A[0])):
```

```
            row.append(A[i][j] + B[i][j])
```

```
        result.append(row)
```

```
    return result
```

```
def matrix_multiplication(A, B):
```

```
    rows_A = len(A)
```

```
    cols_A = len(A[0])
```

```
    cols_B = len(B[0])
```

```
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
```

```
    for i in range(rows_A):
```

```
    for j in range(cols_B):
        for k in range(len(B)):
            result[i][j] += A[i][k] * B[k][j]
return result
```

```
print("Enter the main matrix:")
main_matrix = create_2d_array()
```

```
while True:
    print("\n2-D Array Operations")
    print("1. Row-wise Sum")
    print("2. Column-wise Sum")
    print("3. Addition with Another Matrix")
    print("4. Multiplication with Another Matrix")
    print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    print("Row-wise sum:", row_sum(main_matrix))
```

```
elif choice == 2:
    print("Column-wise sum:", column_sum(main_matrix))
```

```
elif choice == 3:
    print("Enter another matrix for addition:")
```

```
matrix2 = create_2d_array()
if main_matrix.shape == matrix2.shape:
    result = matrix_addition(main_matrix.tolist(), matrix2.tolist())
    print("Result after addition:")
    for row in result:
        print(row)
else:
    print("Error: Matrices must be the same size.")
```

```
elif choice == 4:
    print("Enter another matrix for multiplication:")
    matrix2 = create_2d_array()
    if main_matrix.shape[1] == matrix2.shape[0]:
        result = matrix_multiplication(main_matrix.tolist(), matrix2.tolist())
        print("Result after multiplication:")
        for row in result:
            print(row)
    else:
        print("Error: Columns of first matrix must equal rows of second.")
```

```
elif choice == 5:
    print("Goodbye!")
    break
```

```
else:
    print("Invalid choice. Try again.")
```



```
cont = input("Do you want to continue? (y/n): ").lower()
```

```
if cont != 'y':
```

```
    print("Goodbye!")
```

```
    break
```

OUTPUT –

```
Enter the main matrix:
Enter the number of rows: 2
Enter the number of columns: 3
Enter the elements row-wise:
Enter element at position [0][0]: 1
Enter element at position [0][1]: 2
Enter element at position [0][2]: 3
Enter element at position [1][0]: 4
Enter element at position [1][1]: 5
Enter element at position [1][2]: 6

2-D Array Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
Enter your choice: 1
Row-wise sum: [6, 15]
Do you want to continue? (y/n): y

2-D Array Operations
1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit
Enter your choice: 2
Column-wise sum: [5, 7, 9]
Do you want to continue? (y/n): y
```

2-D Array Operations

1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit

Enter your choice: 3

Enter another matrix for addition:

Enter the number of rows: 2

Enter the number of columns: 3

Enter the elements row-wise:

Enter element at position [0][0]: 9

Enter element at position [0][1]: 8

Enter element at position [0][2]: 7

Enter element at position [1][0]: 6

Enter element at position [1][1]: 6

Enter element at position [1][2]: 5

Result after addition:

[10, 10, 10]

[10, 11, 11]

Do you want to continue? (y/n): y

2-D Array Operations

1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit

Enter your choice: 4

Enter another matrix for multiplication:

Enter the number of rows: 3

Enter the number of columns: 2

Enter the elements row-wise:

Enter element at position [0][0]: 1

Enter element at position [0][1]: 2

Enter element at position [1][0]: 3

Enter element at position [1][1]: 4

Enter element at position [2][0]: 5

Enter element at position [2][1]: 6

Result after multiplication:

[22, 28]

[49, 64]

Do you want to continue? (y/n): y

2-D Array Operations

1. Row-wise Sum
2. Column-wise Sum
3. Addition with Another Matrix
4. Multiplication with Another Matrix
5. Exit

Enter your choice: 5

Goodbye!